

Análisis Léxico y Símbolos

Especificación Léxica y Diseño del Lenguaje Base

DRA. MELISSA BRIGITTHE HINOJOSA CABELLO

Universidad Nacional Autónoma de México

Facultad de Ingeniería

22 de Septiembre de 2026





El análisis léxico es la primera interacción real del compilador con el programa escrito en lenguaje de alto nivel: recibe una secuencia de instrucciones, tal como están escritas en el código fuente y **su objetivo es darle forma** a dichas instrucciones.

- ▶ Agrupar los caracteres en tokens.
- ▶ Identificar qué partes del programa corresponden a palabras clave, identificadores, operadores, etc.
- ▶ Eliminar información irrelevante para las fases posteriores.

El resultado de esta fase es una secuencia de tokens e información básica asociada a cada token (tipo, valor, posición en el código) que será utilizada por el análisis sintáctico para verificar la estructura del programa.

Especificación léxica



En un compilador, la **especificación léxica** describe formalmente los tipos de tokens que pueden aparecer en un programa fuente.

Para ello se definen patrones que permiten reconocer secuencias válidas de caracteres y clasificarlas en distintas categorías léxicas. Estas definiciones se expresan normalmente mediante expresiones regulares, ya que los tokens de un lenguaje de programación forman lenguajes regulares.

Ejemplo de especificación léxica:

- ▶ NUMBER: $[0-9]^+$
- ▶ IDENTIFIER: $[a-zA-Z][a-zA-Z0-9]^*$
- ▶ PLUS: "+"
- ▶ MINUS: "-"
- ▶ ASSIGN: "="

Ejemplo de cadena: $x = 10 + y$

→ El analizador léxico leería la cadena carácter por carácter y produciría la secuencia de tokens:

- | | |
|-----------------|-----------------|
| ▶ IDENTIFIER(x) | ▶ PLUS |
| ▶ ASSIGN | ▶ IDENTIFIER(y) |
| ▶ NUMBER(10) | |



Durante el análisis léxico, el programa fuente se procesa carácter por carácter y las secuencias que coinciden con los patrones definidos se agrupan para formar tokens, que posteriormente serán utilizados por el analizador sintáctico.

→ La especificación léxica es básicamente una lista de expresiones regulares que describen todos los tokens del lenguaje.

¿Qué es un token?

Token

Es una unidad léxica que representa una categoría de elementos dentro de un lenguaje de programación y puede describirse como una tupla $\langle \textit{tipo}, \textit{atributo} \rangle$, donde *tipo* indica la categoría léxica y *atributo* el valor del lexema.



En la definición anterior hablamos de categorías o clases léxicas y lexemas, ¿a qué nos referimos con ello?

Las **clases léxicas** son las categorías en las que el analizador léxico o **lexer** agrupa los tokens que comparten una misma estructura o función dentro del lenguaje. Cada clase léxica se define mediante un patrón que describe la forma de las cadenas válidas que pertenecen a esa categoría.

→ Los tokens de una misma clase léxica comparten una misma estructura sintáctica o que cumplen una función similar dentro del lenguaje de programación.



Entre las clases léxicas más comunes en los lenguajes de programación se encuentran:

- ▶ **Identificadores**, que representan nombres de variables o funciones.
- ▶ **Números**, que representan valores numéricos.
- ▶ **Palabras reservadas**, que corresponden a palabras con significado especial en el lenguaje.
- ▶ **Operadores**, que indican operaciones a realizar.
- ▶ **Delimitadores**, que se utilizan para estructurar el programa.



Un **patrón** se expresa normalmente mediante expresiones regulares y corresponde a una descripción formal de cómo se ve una categoría de palabras del lenguaje. Es decir, describe el conjunto de cadenas que pertenecen a una determinada clase léxica.

Ejemplo: `letra (letra | dígito)*`
¿Qué nos indica este patrón? ¿Para qué sirve?

Este patrón define la forma de los identificadores e indica que:

Ejemplos:

- ▶ El primer carácter debe ser una letra.
- ▶ Después pueden venir cero o más letras o dígitos.



Un **lexema** es la secuencia concreta de caracteres que aparece en el programa fuente y que coincide con un patrón específico.

Entonces, retomando el patrón para identificadores del ejemplo anterior (letra (letra | dígito)*), cadenas como x, contador, y var1, son lexemas que pertenecen a la clase léxica de identificadores.

Recordatorio:

Patrón → Regla formal que describe una clase de cadenas.

vs.

Lexema → Secuencia concreta de caracteres encontrada.

vs.

Token → Categoría léxica asignada al lexema.



Cuando el analizador léxico procesa el texto de entrada, puede suceder que varias reglas coincidan con un mismo prefijo de la cadena. Para resolver esta situación, los analizadores léxicos aplican la **regla de mayor coincidencia**, que establece que el lexer debe seleccionar el lexema más largo posible que coincida con algún patrón definido. Ejemplo:

Supongamos que el lenguaje tiene los operadores $<$, $>$, $=$, $<=$, $>=$
Y se recibe como entrada ' $>=$ '

¿Cuáles serían las posibles coincidencias?

$>$, $>=$

¿Cuál elige el lexer?

El lexer elige ' $>=$ ' porque es el operador más largo.



No obstante, en algunos casos, varios patrones pueden coincidir con la misma cadena de entrada. Cuando esto ocurre, los analizadores léxicos utilizan además una **prioridad de reglas** para decidir cuál token producir. Generalmente, la prioridad se establece mediante el orden en que las reglas aparecen en la especificación léxica. Ejemplo:

Supongamos que el lenguaje tiene las reglas

if → palabra reservada

identificador → letra (letra | dígito)*

Y se recibe como entrada '*if*'

¿Cuál elige el lexer?

El lexer debe reconocerla como palabra reservada y no como identificador, debido a que la regla correspondiente tiene mayor prioridad.



Antes de analizar la sintaxis o la semántica, un compilador debe entender el texto del programa, y esto es justo lo que hace el lexer. El análisis léxico consiste en:

- ▶ Leer el programa carácter por carácter.
- ▶ Agrupar caracteres en unidades llamadas lexemas.
- ▶ Clasificar esas unidades como tokens.

Para que esto sea posible, primero debemos definir los elementos fundamentales del lenguaje, como el alfabeto, los tokens, operadores y palabras reservadas.

Diseño del lenguaje base



El **alfabeto** es el conjunto de símbolos válidos del lenguaje, es decir, define qué caracteres pueden aparecer en el código fuente.

Ejemplos de símbolos:

- ▶ Letras: a–z, A–Z
- ▶ Dígitos: 0–9
- ▶ Palabras clave: *if*, *while*, *return*, *class*, ...
- ▶ Operadores: +, −, *, /, <, >, =, ==
- ▶ Delimitadores: () { } ;

→ Entonces, un lenguaje se construye sobre un alfabeto, que generalmente se representa como Σ .

Recordatorio:

El analizador léxico no trabaja directamente con las palabras, sino con tokens (categorías léxicas).



Los **literales** representan valores que aparecen directamente en el código y no cambian durante la ejecución del programa. Los literales son importantes porque el compilador debe reconocerlos y convertirlos en valores internos.

Ejemplos:

- edad = 20
- pi = 3.141592
- flag = true

Los **operadores** representan acciones que se realizan sobre los datos y se clasifican normalmente en aritméticos, relacionales, lógicos y de asignación.

Las **palabras reservadas** son palabras que tienen significado especial dentro del lenguaje y se usan para definir estructuras del programa, por lo que no pueden utilizarse como identificadores.